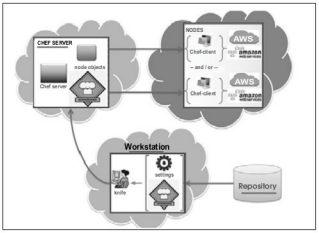

Figure 1: Chef's components

Puppet

Another SCM tool commonly used is Puppet. It was first introduced in 2005 as an open source configuration management tool. It is written in Ruby. This CM system allows defining the state of the IT infrastructure, and then automatically enforces the correct state.

It is used to manage the configuration of UNIX-like and Microsoft Windows systems. The user describes the system's resources and their state, either by using Puppet's declarative language or a Ruby DSL. This information is stored in files known as Puppet manifests. It discovers system information through a utility called Facter and compiles it into a system-specific catalogue containing resources and their dependencies, which are applied against the target systems.

When an application is deployed or is working with internal software, then Puppet automates every step of the delivery process—from the provisioning of physical and virtual machines, to reporting code development through testing, updates and product release. It ensures stability, reliability and consistency. It also aids close collaboration between developers and sys admins, enabling more efficient delivery of code.

Once Puppet is installed, every node (physical server, device or virtual machine) in the infrastructure will have a Puppet agent installed on it. There will also be a server known as Puppet Master. Implementation takes place during Puppet runs as per the steps shown in Figure 2.

CFEngine

CFEngine is one of the most popular open source and fully distributed CM systems and provides automated configuration compute resources. In addition, it provides cohesive management and monitoring of physical or virtual machines, tablets, embedded devices, cloud and mobile smartphones, and can be called an IT infrastructure automation framework. It is available in open source and commercial flavours. CFEngine is based on policies, which are used to deploy or patch up systems and are made up of promises. The latter are verified frequently to ensure full compliance with a desired

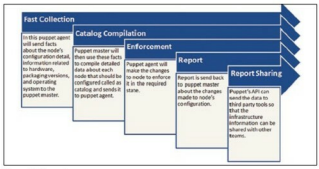

Figure 2: Puppet

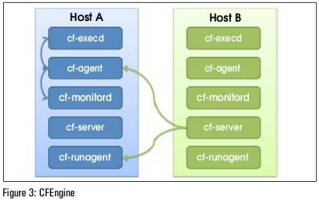

Figure 3: CFEngine

state. Bringing resources into the desired state for compliance with policies is known as convergence. All policies are made available to all resources with the use of the policy server. This server monitors compute resources across the IT network, based on policies. Client hosts need to ensure that they continuously fetch the latest and most up-to-date policies from the policy server and that they remain compliant based on the set of instructions available in the policy language.

Components in the CFEngine are independent agents and are the basis of automation.

cf-execd	This is the scheduling daemon for cf-agent. Its responsibility is to execute and collect the output of cf-agent and send email notifications to the configured email addresses.
cf-serverd	cf-serverd acts as a file server for operations such as remote file copying. It allows an authorised cf-runagent to start a cf-agent run. It provides a line of communication between the hosts.
cf-monitord	This is the monitoring daemon. It keeps the promises available in common and monitor bundles.
cf-agent	The main work of cf-agent is to evaluate policies and make changes to the resources. It is the core of the CFEngine that manipulates resources based on policies.
cf-runagent	This is a helper program to run the cf-agent. It commands cf-serverd to execute the cf-agent with its active policy and hence is responsible for the instant deployment of updated policy.